BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 8

Roll no:56 PRN No: 12412988 Name of Student: Arya Sawant

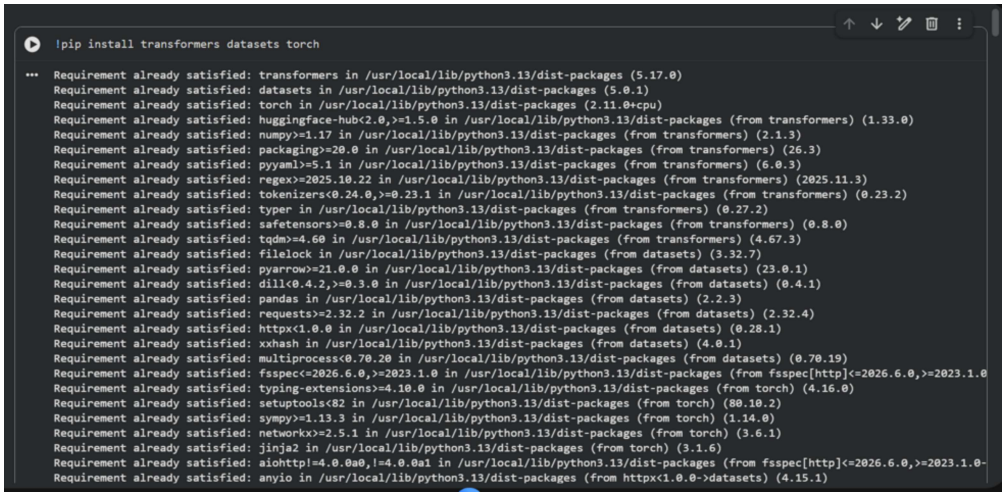
Subject Name : Deep Learning

1. Aim: Implement Pre-trained BERT model for sentiment analysis or text classification on a sample dataset

2. Algorithm :

1. Start.
2. Install and import the required libraries such as PyTorch, Transformers, and Datasets.
3. Load the IMDB movie review dataset containing positive and negative reviews.
4. Split/select the dataset into training and testing data.
5. Load the pre-trained BERT tokenizer and tokenize the text reviews.
6. Load the pre-trained BERT model with two output classes: Positive and Negative.
7. Fine-tune the BERT model using the training dataset.
8. Evaluate the trained model using the testing dataset.
9. Calculate the classification accuracy.
10. Give new text reviews as input to the trained model.
11. Predict whether each review is Positive or Negative.
12. Display the predictions and accuracy.
13. Stop.

Code:



```
!pip install transformers datasets torch

Requirement already satisfied: transformers in /usr/local/lib/python3.13/dist-packages (5.17.0)
Requirement already satisfied: datasets in /usr/local/lib/python3.13/dist-packages (5.0.1)
Requirement already satisfied: torch in /usr/local/lib/python3.13/dist-packages (2.11.0+cpu)
Requirement already satisfied: huggingface-hub<2.0,>=1.5.0 in /usr/local/lib/python3.13/dist-packages (from transformers) (1.33.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.13/dist-packages (from transformers) (2.1.3)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from transformers) (26.3)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.13/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex>=2025.10.22 in /usr/local/lib/python3.13/dist-packages (from transformers) (2025.11.3)
Requirement already satisfied: tokenizers<0.24.0,>=0.23.1 in /usr/local/lib/python3.13/dist-packages (from transformers) (0.23.2)
Requirement already satisfied: typer in /usr/local/lib/python3.13/dist-packages (from transformers) (0.27.2)
Requirement already satisfied: safetensors>=0.8.0 in /usr/local/lib/python3.13/dist-packages (from transformers) (0.8.0)
Requirement already satisfied: tqdm>=4.60 in /usr/local/lib/python3.13/dist-packages (from transformers) (4.67.3)
Requirement already satisfied: filelock in /usr/local/lib/python3.13/dist-packages (from datasets) (3.32.7)
Requirement already satisfied: pyarrow>=21.0.0 in /usr/local/lib/python3.13/dist-packages (from datasets) (23.0.1)
Requirement already satisfied: dill<0.4.2,>=0.3.0 in /usr/local/lib/python3.13/dist-packages (from datasets) (0.4.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (from datasets) (2.2.3)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.13/dist-packages (from datasets) (2.32.4)
Requirement already satisfied: httpx<1.0.0 in /usr/local/lib/python3.13/dist-packages (from datasets) (0.28.1)
Requirement already satisfied: xxhash in /usr/local/lib/python3.13/dist-packages (from datasets) (4.0.1)
Requirement already satisfied: multiprocess<0.70.20 in /usr/local/lib/python3.13/dist-packages (from datasets) (0.70.19)
Requirement already satisfied: fsspec<=2026.6.0,>=2023.1.0 in /usr/local/lib/python3.13/dist-packages (from fsspec[http]<=2026.6.0,>=2023.1.0
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.13/dist-packages (from torch) (4.16.0)
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.13/dist-packages (from torch) (80.10.2)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.13/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.13/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.13/dist-packages (from torch) (3.1.6)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.13/dist-packages (from fsspec[http]<=2026.6.0,>=2023.1.0-
Requirement already satisfied: anyio in /usr/local/lib/python3.13/dist-packages (from httpx<1.0.0->datasets) (4.15.1)
```

```
!pip install -U datasets huggingface_hub transformers
```

```
Requirement already satisfied: datasets in /usr/local/lib/python3.13/dist-packages (5.0.1)
Requirement already satisfied: huggingface_hub in /usr/local/lib/python3.13/dist-packages (1.33.0)
Collecting huggingface_hub
  Using cached huggingface_hub-2.0.0-py3-none-any.whl.metadata (16 kB)
Requirement already satisfied: transformers in /usr/local/lib/python3.13/dist-packages (5.17.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.13/dist-packages (from datasets) (3.32.7)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.13/dist-packages (from datasets) (2.1.3)
Requirement already satisfied: pyarrow>=21.0.0 in /usr/local/lib/python3.13/dist-packages (from datasets) (23.0.1)
Requirement already satisfied: dill<0.4.2,>=0.3.0 in /usr/local/lib/python3.13/dist-packages (from datasets) (0.4.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (from datasets) (2.2.3)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.13/dist-packages (from datasets) (2.32.4)
Requirement already satisfied: httpx<1.0.0 in /usr/local/lib/python3.13/dist-packages (from datasets) (0.28.1)
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.13/dist-packages (from datasets) (4.67.3)
Requirement already satisfied: xxhash in /usr/local/lib/python3.13/dist-packages (from datasets) (4.0.1)
Requirement already satisfied: multiprocess<0.70.20 in /usr/local/lib/python3.13/dist-packages (from datasets) (0.70.19)
Requirement already satisfied: fsspec<=2026.6.0,>=2023.1.0 in /usr/local/lib/python3.13/dist-packages (from fsspec[http]<=2026.6.0,>=2023.1.0
Requirement already satisfied: packaging in /usr/local/lib/python3.13/dist-packages (from datasets) (26.3)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.13/dist-packages (from datasets) (6.0.3)
Requirement already satisfied: click<9.0.0,>=8.4.2 in /usr/local/lib/python3.13/dist-packages (from huggingface_hub) (8.5.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.6.0 in /usr/local/lib/python3.13/dist-packages (from huggingface_hub) (1.6.0)
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.13/dist-packages (from huggingface_hub) (4.16.0)
Requirement already satisfied: regex>=2025.10.22 in /usr/local/lib/python3.13/dist-packages (from transformers) (2025.11.3)
Requirement already satisfied: tokenizers<0.24.0,>=0.23.1 in /usr/local/lib/python3.13/dist-packages (from transformers) (0.23.2)
Requirement already satisfied: typer in /usr/local/lib/python3.13/dist-packages (from transformers) (0.27.2)
Requirement already satisfied: safetensors>=0.8.0 in /usr/local/lib/python3.13/dist-packages (from transformers) (0.8.0)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.13/dist-packages (from fsspec[http]<=2026.6.0,>=2023.1.0-
```

```
import datasets
import huggingface_hub

print("Datasets:", datasets.__version__)
print("Hugging Face Hub:", huggingface_hub.__version__)
```

```
Datasets: 5.0.1
Hugging Face Hub: 1.33.0
```

```
from datasets import load_dataset # Import function to load datasets

# Load the IMDb dataset from Hugging Face Datasets. This dataset is commonly used for sentiment analysis.
dataset = load_dataset("stanfordnlp/imdb")

print(dataset) # Print the dataset structure (train and test splits)
```

```
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    unsupervised: Dataset({
        features: ['text', 'label'],
        num_rows: 50000
    })
})
```

```
# Select a subset of the training and testing data for faster processing during demonstration.
# .shuffle(seed=42) shuffles the dataset with a fixed seed for reproducibility.
# .select(range(num_samples)) picks the first 'num_samples' records after shuffling.
train_data = dataset["train"].shuffle(seed=42).select(range(2000))
test_data = dataset["test"].shuffle(seed=42).select(range(500))

print("Training samples:", len(train_data))
print("Testing samples:", len(test_data))
```

```
Training samples: 2000
Testing samples: 500
```

```
# Load the pre-trained tokenizer for 'bert-base-uncased'.
# A tokenizer converts text into numerical IDs that a model can understand.
# 'bert-base-uncased' is a popular version of the BERT model that does not distinguish between uppercase and lowercase letters.
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
```

```
# Define a tokenization function that takes examples (text) and processes them.
def tokenize_function(examples):
    return tokenizer(
        examples["text"], # Input text from the dataset
        padding="max_length", # Pad sequences to the maximum length of the model's input
        truncation=True, # Truncate sequences longer than max_length
        max_length=128 # Set the maximum sequence length for the model
    )

# Apply the tokenization function to both the training and testing datasets.
# batched=True processes multiple examples at once, which is more efficient.
```

```python
# Apply the tokenization function to both the training and testing datasets.
# batched=True processes multiple examples at once, which is more efficient.
train_data = train_data.map(tokenize_function, batched=True)
test_data = test_data.map(tokenize_function, batched=True)
```

```python
# Set the format of the datasets to "torch" tensors. This makes them compatible with PyTorch.
# We specify the columns that will be used by the model: input_ids, attention_mask, and label.
train_data.set_format(
    type="torch",
    columns=["input_ids", "attention_mask", "label"]
)

test_data.set_format(
    type="torch",
    columns=["input_ids", "attention_mask", "label"]
)
```

```python
# Load a pre-trained BERT model for sequence classification.
# 'bert-base-uncased' is the pre-trained model checkpoint.
# num_labels=2 specifies that this is a binary classification task (e.g., positive/negative sentiment).
model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2
)
```

```
Loading weights: 100% 199/199 [00:00<00:00, 2059.91it/s]
```

```python
# Load a pre-trained BERT model for sequence classification.
# 'bert-base-uncased' is the pre-trained model checkpoint.
# num_labels=2 specifies that this is a binary classification task (e.g., positive/negative sentiment).
model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2
)
```

```
Loading weights: 100% 199/199 [00:00<00:00, 2059.91it/s]
[transformers] BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                         | Status     |
--------------------------------------------+------------+-
cls.predictions.transform.dense.bias        | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight  | UNEXPECTED |
cls.seq_relationship.weight                 | UNEXPECTED |
cls.predictions.bias                        | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias    | UNEXPECTED |
cls.seq_relationship.bias                   | UNEXPECTED |
cls.predictions.transform.dense.weight      | UNEXPECTED |
classifier.weight                           | MISSING    |
classifier.bias                             | MISSING    |

Notes:
- UNEXPECTED:    can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING:  those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
```

```python
# Create DataLoader objects for the training and testing datasets.
# DataLoaders help to iterate over the dataset in batches, shuffle the data, and handle parallelism.
train_loader = DataLoader(
```

```python
# Create DataLoader objects for the training and testing datasets.
# DataLoaders help to iterate over the dataset in batches, shuffle the data, and handle parallelism.
train_loader = DataLoader(
    train_data, # Our tokenized and formatted training data
    batch_size=8, # Number of samples per batch during training
    shuffle=True # Shuffle the data at each epoch for better generalization
)

test_loader = DataLoader(
    test_data, # Our tokenized and formatted testing data
    batch_size=8 # Number of samples per batch during evaluation
)
```

```python
# Determine which device (GPU 'cuda' or CPU) is available for computations.
# Using a GPU significantly speeds up training for deep learning models.
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("Using device:", device)

# Move the model to the selected device.
model.to(device)
```

```
Using device: cpu
BertForSequenceClassification(
  (bert): BertModel(
    (embeddings): BertEmbeddings(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
```

```
Using device: cpu
BertForSequenceClassification(
  (bert): BertModel(
    (embeddings): BertEmbeddings(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (token_type_embeddings): Embedding(2, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (encoder): BertEncoder(
      (layer): ModuleList(
        (0-11): 12 x BertLayer(
          (attention): BertAttention(
            (self): BertSelfAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (output): BertSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
          (intermediate): BertIntermediate(
            (dense): Linear(in_features=768, out_features=3072, bias=True)
            (intermediate_act_fn): GELUActivation()
          )
          (output): BertOutput(
            (dense): Linear(in_features=3072, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
    )
    (pooler): BertPooler(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (activation): Tanh()
    )
  )
  (dropout): Dropout(p=0.1, inplace=False)
  (classifier): Linear(in_features=768, out_features=2, bias=True)
)
```

```
# Initialize the optimizer. AdamW is a popular choice for BERT-like models.
# It updates the model's weights during training to minimize the loss function.
# lr (learning rate) controls the step size of the weight updates.
optimizer = torch.optim.AdamW(
    model.parameters(), # Parameters (weights) of the model to be optimized
    lr=2e-5 # Learning rate
)
```

```
# Set the model to training mode. This enables features like Dropout.
model.train()

# Loop through training epochs (one full pass over the training data).
```

```
# Set the model to training mode. This enables features like Dropout.
model.train()

# Loop through training epochs (one full pass over the training data).
for epoch in range(1):
    total_loss = 0 # Initialize total loss for the current epoch

    # Iterate over batches from the training DataLoader.
    for batch in train_loader:

        # Move input data (input_ids, attention_mask, labels) to the appropriate device (CPU/GPU).
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["label"].to(device)

        optimizer.zero_grad() # Clear previous gradients before computing new ones

        # Perform a forward pass: feed inputs to the model to get outputs.
        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels # Provide labels to compute the loss
        )

        loss = outputs.loss # Extract the loss from the model's outputs
        total_loss += loss.item() # Add the current batch's loss to the total

        loss.backward() # Perform a backward pass to compute gradients
        optimizer.step() # Update model weights based on the computed gradients
```

```
        # Print the average training loss for the current epoch.
        print(
            "Epoch:", epoch + 1,
            "Training Loss:", total_loss / len(train_loader)
        )
```

```
Epoch: 1 Training Loss: 0.4995568874776363
```

```
# Set the model to evaluation mode. This disables features like Dropout that are specific to training.
model.eval()

correct = 0 # Initialize counter for correctly predicted samples
total = 0 # Initialize counter for total samples

# Disable gradient calculations during evaluation to save memory and speed up computation.
with torch.no_grad():

    # Iterate over batches from the test DataLoader.
    for batch in test_loader:

        # Move input data to the appropriate device.
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["label"].to(device) # Ground truth labels

        # Perform a forward pass without labels (as we are evaluating, not training).
        outputs = model(
            input_ids=input_ids,
```

```
                        _

        # Move input data to the appropriate device.
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["label"].to(device) # Ground truth labels

        # Perform a forward pass without labels (as we are evaluating, not training).
        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask
        )

        # Get the predicted class by finding the index of the maximum logit (output score).
        predictions = torch.argmax(
            outputs.logits,
            dim=1 # Predict along the dimension representing classes
        )

        # Update correct and total counts.
        correct += (predictions == labels).sum().item() # Count matching predictions
        total += labels.size(0) # Add batch size to total count

# Calculate accuracy: (correct predictions / total predictions).
accuracy = correct / total

print("Test Accuracy:", accuracy)
```

```
Test Accuracy: 0.854
```

```
    # Define example movie reviews for inference.
    reviews = [
        "This movie was amazing and I really enjoyed it.",
        "The movie was boring and disappointing."
    ]

    # Tokenize the new reviews, padding and truncating them to the model's expected input format.
    # return_tensors="pt" ensures the output is PyTorch tensors.
    inputs = tokenizer(
        reviews,
        padding=True,
        truncation=True,
        max_length=128,
        return_tensors="pt"
    )

    # Move the tokenized inputs to the same device as the model.
    inputs = {
        key: value.to(device)
        for key, value in inputs.items()
    }

    # Set the model to evaluation mode for inference.
    model.eval()

    # Perform inference without gradient calculation.
    with torch.no_grad():
        outputs = model(**inputs) # Pass the tokenized inputs to the model
```

```python
# Perform inference without gradient calculation.
with torch.no_grad():
    outputs = model(**inputs) # Pass the tokenized inputs to the model

# Get the predicted class (sentiment) for each review.
predictions = torch.argmax(
    outputs.logits,
    dim=1
)

# Iterate through reviews and their corresponding predictions to display the sentiment.
for review, prediction in zip(reviews, predictions):

    if prediction.item() == 1:
        sentiment = "Positive"
    else:
        sentiment = "Negative"

    print("Review:", review)
    print("Sentiment:", sentiment)
    print()
```

```
Review: This movie was amazing and I really enjoyed it.
Sentiment: Positive

Review: The movie was boring and disappointing.
Sentiment: Negative
```